handling will entangle the functionality of processing log files with database connection recovery logic. Even if you found a spot to re-create the connection, you'd have to be very careful that other threads didn't use the faulty connection while you were trying to replace it with a new one; otherwise, some rows would be lost.

Also, communicating exceptions between threads isn't a standard feature; you have to build this yourself, which isn't trivial. Let's look at the fault-tolerance requirements to see if this approach stands a chance:

- *Fault isolation*—Isolation is made difficult by the fact that many threads can throw exceptions at the same time. You have to add a locking mechanism. It's hard to remove the faulty connection from the chain of objects: the application would have to be rewritten to get this to work. There's no standard support for cutting off the use of the connection in the future, so this needs to be built into the objects manually with some level of indirection.
- *Structure*—The structure that exists between objects is simple and direct. Every object possibly refers to other objects, forming a graph; you can't replace an object in the graph at runtime. You have to create a more involved structure (again, with a level of indirection between the objects).
- *Redundancy*—When an exception is thrown, it goes up the call stack. You may miss the context for deciding which redundant component to use or lose the context of which input data to continue with, as seen in the preceding example.